	Politechnika Bydgoska im. J. J. Śniadeckich Wydział Telekomunikacji, Informatyki i Elektrotechniki		
Przedmiot	Zaawansowane programowanie obiektowe		
Prowadzący	dr inż. Damian Szczegielniak		
Temat	Implementacja usługi sieciowej typu REST		
Student	Jakub Ziółkowski, grupa 1, Informatyka stosowana, IV semestr, studia stacjonarne		
Nr ćw.	4	Data wykonania	20.06.2026 r.
Ocena		Data oddania spr.	14.07.2026 r.

Spis treści

1. Cel laboratorium.....	2
2. Rozwiązanie.....	3
2.1. Ogólny opis architektury usługi REST i Spring Boot	3
2.2. Konfiguracja bazy danych PostgreSQL	3
2.3. Struktura projektu Gradle i konfiguracja application.properties	4
2.4. Zabezpieczenie usługi - klasa SecurityConfig i Basic Authentication	6
2.5. Model danych i mapowanie obiektowo-relacyjne (klasy Projekt, Zadanie, Student)	7
2.7. Warstwa logiki biznesowej i serwisy aplikacyjne	10
2.8. Kontrolery REST - klasa ProjektRestController.....	12
2.9. Konfiguracja mechanizmu rejestracji (w pliku logback-spring.xml)	14
2.10. Testowanie REST API za pomocą programu Postman.....	14
2.11. Dokumentacja usługi za pomocą interfejsu Swagger UI	16

2.12. Testy integracyjne i jednostkowe	18
2.13. Tworzenie wersji uruchomieniowej i automatyzacja uruchamiania	20
3. Wnioski końcowe/Podsumowanie	23

1. Cel laboratorium

Głównym celem laboratorium było zaprojektowanie, zaimplementowanie oraz przetestowanie usługi sieciowej typu REST (Representational State Transfer) przy użyciu zaawansowanych możliwości platformy Spring Boot. Laboratorium uwzględniło też przygotowanie wersji uruchomieniowej usługi w postaci pliku JAR oraz automatyzację jej uruchamiania za pomocą skryptów wsadowych.

Ćwiczenie to zakładało zapoznanie się ze standardami budowy interfejsów programistycznych (API), automatycznym mapowaniem obiektowo-relacyjnym zapewnianym przez specyfikację Jakarta Persistence (JPA) oraz framework Hibernate, a także integracją aplikacji z zewnętrznym, niezależnym serwerem relacyjnej bazy danych PostgreSQL. Zadanie wymagało zachowania podziału architektonicznego oraz wdrożenia mechanizmów uwierzytelniania i autoryzacji zasobów za pomocą modułu Spring Security.

Szczególną uwagę zwróciłem na mechanizm optymalizacji wydajności serwera sieciowego, jakim są wątki wirtualne. Ważnym aspektem było również nauczenie się automatycznego generowania dokumentacji technicznej API zgodnie ze specyfikacją OpenAPI 3 / Swagger, automatyzacja procesu testowego przez pisanie testów integracyjnych i jednostkowych przy użyciu MockMVC oraz Mockito, a także wdrożenie systemu logowania zdarzeń z pomocą biblioteki Logback.

2. Rozwiązanie

2.1. Ogólny opis architektury usługi REST i Spring Boot

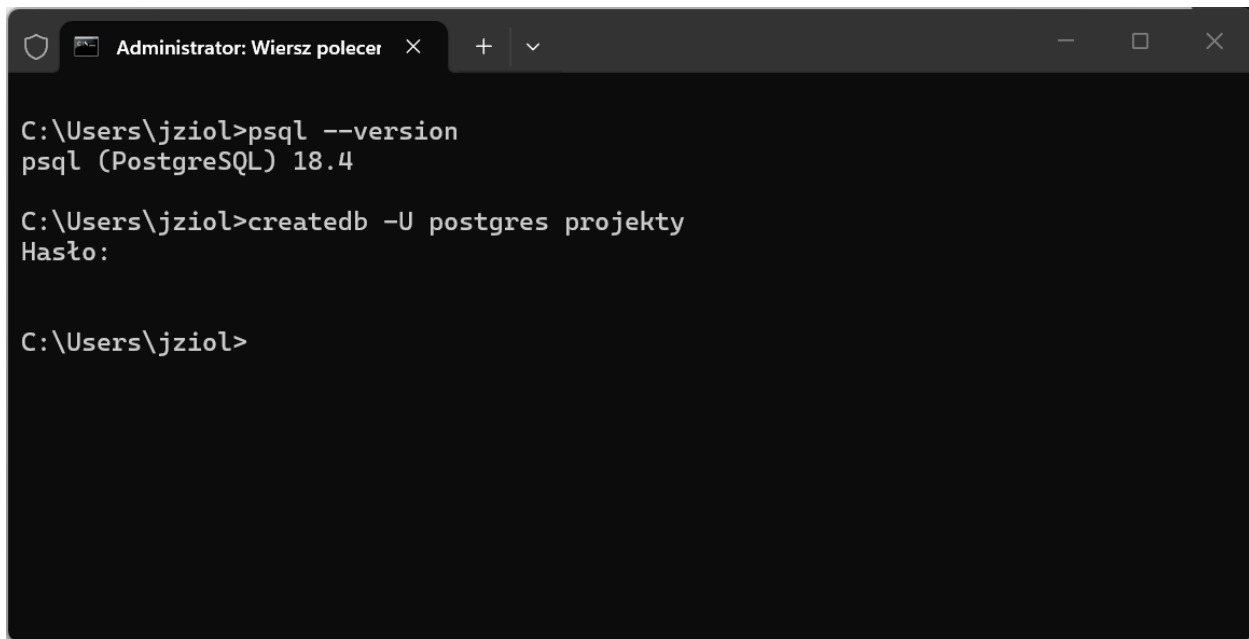
Architektura trójwarstwowa (Controller-Service-Repository) jest oparta na platformie Spring Boot oraz wbudowanym serwerze Apache Tomcat. Usługa Rest API komunikuje się z klientami za pomocą protokołu HTTP i wymienia strukturalne dane w formacie JSON. Wszystkie zasoby aplikacji (Projekty, Zadania, Studenci) są identyfikowane za pomocą unikalnych adresów URI, a operacje na nich są określone przez standardowe metody protokołu HTTP: GET (pobieranie zasobów), POST (tworzenie nowych struktur), PUT (aktualizacja istniejących rekordów) oraz DELETE (usuwanie danych z bazy).

Warstwa prezentacji sieciowej (kontrolery REST) jest oddzielona od bezpośredniego dostępu do bazy, wymuszając przepływ danych przez warstwę logiki biznesowej (serwisy aplikacyjne). Warstwa logiki biznesowej odpowiada za zarządzanie transakcjami, walidację danych oraz koordynację wywoływania poszczególnych repozytoriów. Warstwa persystencji (repozytoria JPA) automatyzuje generowanie zapytań bazodanowych na podstawie zdefiniowanych metod interfejsu oraz automatycznie mapuje tabele relacyjne na obiekty Javy, dzięki czemu nie ma potrzeby ręcznego pisania niskopoziomowych kwerend SQL.

2.2. Konfiguracja bazy danych PostgreSQL

W odróżnieniu od poprzednich laboratoriów wykorzystujących bazę wbudowaną w pamięci lub plik lokalny (HSQLDB), projekt tego laboratorium został zintegrowany z systemem zarządzania relacyjnymi bazami danych PostgreSQL. Proces instalacyjny PostgreSQL 17 w systemie Windows obejmował instalację serwera, narzędzi wiersza poleceń (Command Line Tools) oraz graficznego interfejsu pgAdmin 4. Podczas konfiguracji instalatora zdefiniowałem domyślnego administratora bazy o nazwie 'postgres' wraz z takim samym hasłem uwierzytelniającym.

Dopisałem ścieżkę do podkatalogu bin instalacji PostgreSQL (C:\Program Files\PostgreSQL\17\bin) do systemowej zmiennej środowiskowej Path. Pozwoliło to wywoływać program psql oraz narzędzia administracyjne z dowolnej lokalizacji w konsoli Wiersza polecenia (cmd). Następnie utworzyłem nową bazę danych o nazwie 'projekty' poleceniem 'createdb -U postgres projekty'.



```
C:\Users\jziol>psql --version
psql (PostgreSQL) 18.4

C:\Users\jziol>createdb -U postgres projekty
Hasło:

C:\Users\jziol>
```

Screen 1 - w windowsowym Wierszu polecenia (cmd): weryfikacja instalacji PostgreSQL i dostępności narzędzia psql, a także poprawne wykonanie polecenia createdb dla bazy o nazwie projekty

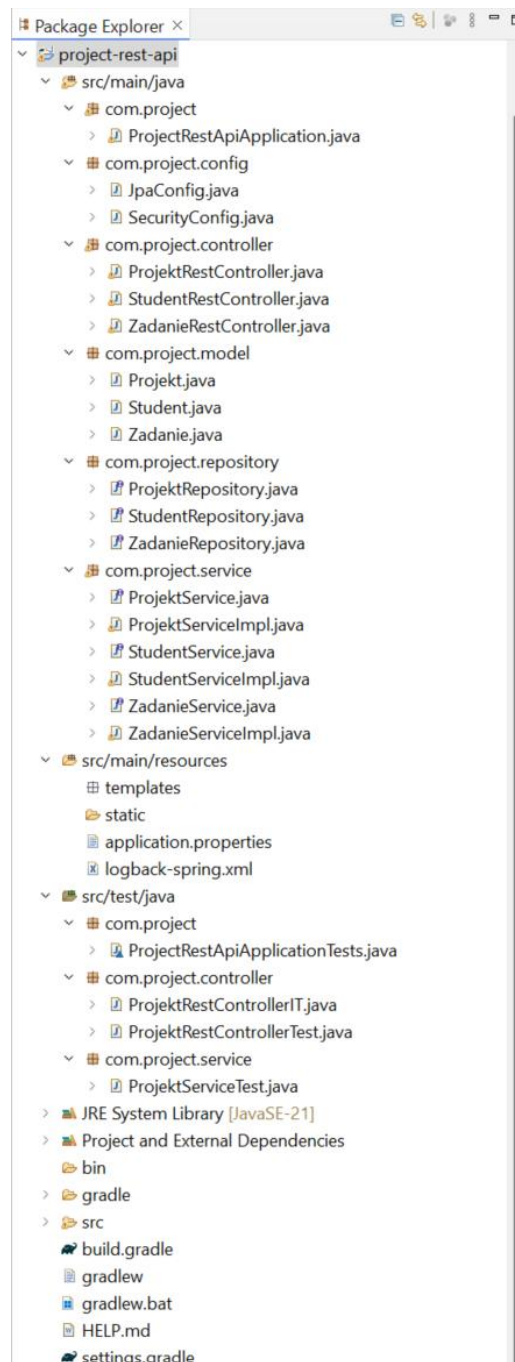
2.3. Struktura projektu Gradle i konfiguracja application.properties

Szkielet projektu o nazwie 'project-rest-api' wygenerowałem za pomocą narzędzia Spring Initializr. Na stronie start.spring.io wybrałem Gradle (z językiem konfiguracji Groovy), wersję języka Java JDK 21 oraz wersję platformy Spring Boot 4.1.0. Wśród zależności użyłem następujące: Spring Web, Spring Security, Validation, Spring Data JPA, PostgreSQL Driver, Lombok oraz Spring Boot DevTools. Wygenerowany projekt zaimportowałem do Eclipse IDE jako Existing Gradle Project.

Edytowałem plik `src/main/resources/application.properties`.

Ten plik zawiera konfigurację sterownika bazy danych, wskazuje lokalny adres URL (`jdbc:postgresql://localhost:5432/projekty`), login oraz hasło administratora. Ponadto, w pliku zdefiniowałem dialekt `PostgreSQLDialect` dla JPA, włączyłem formatowanie oraz logowanie kwerend SQL (`show-sql=true`). Parametr `spring.jpa.hibernate.ddl-auto` ustawiony w tryb 'update' umożliwia automatyczną startową analizę klas encyjnych i modyfikację struktury tabeli PostgreSQL przez Hibernate. W pliku są też zdefiniowane poświadczenia dla domyślnego użytkownika Spring Security (użytkownik i hasło admin), a także aktywowany mechanizm wątków wirtualnych (`spring.threads.virtual.enabled=true`).

Włączenie wątków wirtualnych zapewnia skalowalność aplikacji w trakcie dużego obciążenia sieciowego. Dzięki użyciu wątków wirtualnych, w momencie wystąpienia operacji blokującej (np. oczekiwania na odpowiedź z bazy PostgreSQL), wątek wirtualny jest automatycznie odpinany od fizycznego wątku procesora, umożliwiając mu przetwarzanie kolejnych napływających żądań HTTP. Pozwala to na obsługę setek tysięcy jednoczesnych połączeń bez ryzyka wyczerpania zasobów serwera sieciowego Tomcat.



Screen 2 - widok Package Explorer w Eclipse IDE prezentujący strukturę projektu

```
application.properties x
1 spring.application.name=project-rest-api
2 # Spring DataSource
3 spring.datasource.url=jdbc:postgresql://localhost:5432/projekty
4 spring.datasource.username=postgres
5 spring.datasource.password=postgres
6 spring.datasource.driver-class-name=org.postgresql.Driver
7 # Spring JPA
8 # The SQL dialect makes Hibernate generate better SQL for the chosen database # (Postgres 9.5 and later)
9 spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.PostgreSQLDialect
10 # Validation or export of schema DDL to the database (create, create-drop, validate, update, none)
11 spring.jpa.hibernate.ddl-auto=update
12 # Logging JPA Queries
13 spring.jpa.show-sql=true
14 spring.jpa.properties.hibernate.format_sql=true
15 # Spring Security
16 # HTTP authentication credentials
17 spring.security.user.name=admin
18 spring.security.user.password=admin
19
20 spring.threads.virtual.enabled=true
```

Screen 3 - plik application.properties zawierający parametry bazy, Hibernate oraz włączenia wątków wirtualnych

2.4. Zabezpieczenie usługi - klasa SecurityConfig i Basic Authentication

W celu zabezpieczenia punktów końcowych API przed nieautoryzowanym dostępem, w pakiecie com.project.config utworzyłem klasę SecurityConfig oznaczoną adnotacją @Configuration. Klasa ta definiuje komponent Spring Bean typu SecurityFilterChain. Konfiguracja opiera się na prostym mechanizmie Basic Authentication, w którym poświadczenia użytkownika (login i hasło) są przesyłane w każdym nagłówku żądania HTTP w postaci zakodowanej algorytmem Base64.

Implementacja metody filterChain wyłącza mechanizm ochrony przed atakami CSRF (Cross-Site Request Forgery). Instrukcja authorizeHttpRequests konfiguruje reguły dostępu, wymuszając uwierzytelnienie dla każdego punktu końcowego w aplikacji (anyRequest().authenticated()). Wywołanie httpBasic(Customizer.withDefaults()) aktywuje filtr HTTP Basic, który przetwarza nagłówek 'Authorization' i weryfikuje czy przesłane dane są zgodne z użytkownikiem zdefiniowanym w pliku konfiguracyjnym.

```

SecurityConfig.java x
1 package com.project.config;
2
3 import org.springframework.context.annotation.Bean;
4 import org.springframework.context.annotation.Configuration;
5 import org.springframework.security.config.Customizer;
6 import org.springframework.security.config.annotation.web.builders.HttpSecurity;
7 import org.springframework.security.web.SecurityFilterChain;
8
9 @Configuration
10 public class SecurityConfig {
11
12     @Bean
13     public SecurityFilterChain filterChain (HttpSecurity httpSecurity) throws Exception {
14         return httpSecurity
15             .csrf(csrf -> csrf.disable())
16             .authorizeHttpRequests (auth -> auth.anyRequest().authenticated())
17             .httpBasic (Customizer.withDefaults())
18             .build();
19     }
20 }

```

Screen 4 - klasa SecurityConfig.java definiująca filtr bezpieczeństwa dla protokołu HTTP

2.5. Model danych i mapowanie obiektowo-relacyjne (klasy Projekt, Zadanie, Student)

Warstwę modelową aplikacji zaprojektowałem zgodnie z mapowaniem obiektowo-relacyjnym specyfikacji Jakarta Persistence. W pakiecie `com.project.model` utworzyłem klasy encyjne: `Projekt`, `Zadanie` oraz `Student`, które odwzorowują model relacyjny bazy danych. Zastosowałem wzorzec projektowy Budowniczy za pomocą adnotacji `@Builder`, która pozwala na bardziej czytelne, łańcuchowe tworzenie obiektów.

Klasa `Projekt` zawiera pole unikalnego identyfikatora `projektId` z adnotacjami `@Id` i `@GeneratedValue(strategy=GenerationType.IDENTITY)`, co umożliwia poprawne mapowanie typu `SERIAL` w PostgreSQL. Pola `dataczasUtworzenia` i `dataczasModyfikacji` zostały oznaczone odpowiednio adnotacjami `@CreatedDate` i `@LastModifiedDate`, dzięki temu framework automatycznie zarządza znacznikami czasu.

Aby automatycznie śledzić czas operacji na rekordach, w klasie `JpaConfig` w pakiecie `com.project.config` użyłem mechanizmu audytowania JPA Auditing. Klasa główna została opatrzona adnotacjami `@EnableJpaAuditing` i `@EntityListeners(AuditingEntityListener.class)`.

Relacje pomiędzy tabelami zrealizowałem jako dwukierunkowe powiązania obiektowe. Relacja jeden-do-wielu (1:N) pomiędzy `Projektem` a `Zadaniem` została skonfigurowana za pomocą

adnotacji `@ManyToOne` oraz `@JoinColumn(name='projekt_id')` po stronie klasy `Zadanie` oraz adnotacji `@OneToMany(mappedBy='projekt')` po stronie klasy `Projekt`.

Relacja wiele-do-wielu (M:N) pomiędzy studentami a projektami wymagała utworzenia tabeli pośredniej `'projekt_student'`. W klasie `Projekt` zdefiniowałem zmienną `studenci` typu `Set<Student>` opatrzoną adnotacją `@ManyToMany` oraz `@JoinTable`, określającą nazwy kolumn kluczy obcych (`joinColumns`, `inverseJoinColumns`). Po stronie klasy `Student` relacja jest mapowana parametrem `mappedBy="studenci"`.

Przed listą zadań w klasie `Projekt` oraz przed polem `projekt` w klasie `Zadanie` zastosowałem adnotację `@JsonIgnoreProperties`, aby zapobiec ryzyku wystąpienia nieskończonej pętli (cyklu) podczas automatycznego serializowania obiektów do formatu JSON przez bibliotekę Jackson.

```
Projekt.java x
11 import org.springframework.data.annotation.LastModifiedDate;
12 import org.springframework.data.jpa.domain.support.AuditingEntityListener;
13 import com.fasterxml.jackson.annotation.JsonIgnoreProperties;
14 import lombok.Builder;
15
16 @Entity
17 @EntityListeners(AuditingEntityListener.class)
18 @Table(name = "projekt", indexes = @Index(name = "idx_nazwa_projektu", columnList = "nazwa"))
19 @Builder
20 public class Projekt {
21
22     @Id
23     @GeneratedValue(strategy = GenerationType.IDENTITY)
24     @Column(name = "projekt_id")
25     private Integer projektId;
26
27     @NotBlank(message = "Pole nazwa nie może być puste!")
28     @Size(min = 3, max = 50, message = "Nazwa musi zawierać od {min} do {max} znaków!")
29     @Column(nullable = false, length = 50)
30     private String nazwa;
31
32     @Column(length = 1000)
33     private String opis;
34
35     @CreatedDate
36     @Column(name = "dataczas_utworzenia", nullable = false, updatable = false)
37     private LocalDateTime dataczasUtworzenia;
38
39     @LastModifiedDate
40     @Column(name = "dataczas_modyfikacji", insertable = false)
41     private LocalDateTime dataczasModyfikacji;
42
43     @Column(name = "data_oddania")
44     private LocalDate dataOddania;
45
46     @OneToMany(mappedBy = "projekt")
47     @JsonIgnoreProperties({"projekt"})
48     private List<Zadanie> zadania;
49
50     @ManyToMany
51     @JoinTable(name = "projekt_student",
52               joinColumns = {@JoinColumn(name = "projekt_id")},
53               inverseJoinColumns = {@JoinColumn(name = "student_id")})
54     @JsonIgnoreProperties({"projekty"})
55     private Set<Student> studenci;
56
57     public Projekt() {
58     }
59
60     public Projekt(Integer projektId, String nazwa, String opis, LocalDateTime dataczasUtworze
61                     this.projektId = projektId;
```

Screen 5 - fragment klasy encyjnej `Projekt.java` z wszystkimi opisanymi adnotacjami

2.6. Warstwa persystencji - interfejsy repozytoriów Spring Data JPA

Warstwa persystencji w projekcie opiera się na technologii Spring Data JPA, która upraszcza komunikację z PostgreSQL. W pakiecie `com.project.repository` zdefiniowałem trzy interfejsy: `ProjektRepository`, `StudentRepository` oraz `ZadanieRepository`. Każdy z tych interfejsów dziedziczy po interfejsie `JpaRepository<T, ID>` przyjmując jako typy generyczne typ odpowiedniej encji oraz typ jej klucza głównego (w każdym przypadku `Integer`). Dzięki temu, podczas startu aplikacji Spring automatycznie generuje klasy implementujące te interfejsy, dostarczając gotowe metody obsługi operacji CRUD: `save()`, `findById()`, `findAll()`, `deleteById()`, `count()` i wiele innych.

W interfejsie `ProjektRepository` jest metoda `findByNazwaContainingIgnoreCase(String nazwa, Pageable pageable)`, która generuje zapytanie SQL wykorzystujące operator `LIKE` oraz funkcję `UPPER`, zapewniając filtrowanie danych niezależnie od wielkości liter, z jednoczesną obsługą stronicowania bazy danych. W interfejsie `ZadanieRepository` zastosowałem adnotację `@Query('SELECT z FROM Zadanie z WHERE z.projekt.projekId = :projekId')`, pobierającą tylko te zadania, które należą do konkretnego projektu.

```
ProjektRepository.java ×
1 package com.project.repository;
2
3 import java.util.List;
4 import org.springframework.data.domain.Page;
5 import org.springframework.data.domain.Pageable;
6 import org.springframework.data.jpa.repository.JpaRepository;
7 import com.project.model.Projekt;
8
9 public interface ProjektRepository extends JpaRepository<Projekt, Integer> {
10     Page<Projekt> findByNazwaContainingIgnoreCase(String nazwa, Pageable pageable);
11     List<Projekt> findByNazwaContainingIgnoreCase(String nazwa);
12     // Metoda findByNazwaContainingIgnoreCase definiuje zapytanie
13     // SELECT p FROM Projekt p WHERE upper(p.nazwa) LIKE upper(:%nazwa%)
14 }
```

Screen 6 - interfejs `ProjektRepository.java` z definicjami metod (obsługującej stronicowanie oraz nie)

2.7. Warstwa logiki biznesowej i serwisy aplikacyjne

Logika biznesowa znajduje się w warstwie serwisów aplikacyjnych w pakiecie `com.project.service`. Utworzyłem interfejs `ProjektService` oraz klasę implementacyjną `ProjektServiceImpl` oznaczoną adnotacją `@Service`. Wstrzykiwanie repozytoriów odbywa się przez konstruktor klasy, co sprawia, że nie jest konieczne stosowanie adnotacji `@Autowired` nad polami prywatnymi i ułatwia testy jednostkowe.

Metoda `deleteProjekt(Integer projektId)` w klasie serwisu została opatrzona adnotacją `@Transactional`. W encji nie zdefiniowałem kaskadowego usuwania w JPA przez `CascadeType.REMOVE`, więc próba usunięcia projektu z powiązanymi zadaniami skończyłaby się krytycznym wyjątkiem `SQLException` z naruszeniem więzów integralności klucza obcego. W omawianej metodzie najpierw w pętli pobierane i usuwane są wszystkie zadania powiązane z danym identyfikatorem projektu, a dopiero po tym rekord główny. Dzięki adnotacji obie operacje są wykonywane w ramach jednej transakcji. W przypadku błędu automatycznie wykonywana jest operacja `rollback`.

```

ProjektServiceImpl.java x
1 package com.project.service;
2
3 import java.util.Optional;
4 import org.slf4j.Logger;
5 import org.slf4j.LoggerFactory;
6 import org.springframework.beans.factory.annotation.Autowired;
7 import org.springframework.data.domain.Page;
8 import org.springframework.data.domain.Pageable;
9 import org.springframework.stereotype.Service;
10 import org.springframework.transaction.annotation.Transactional;
11 import com.project.model.Projekt;
12 import com.project.model.Zadanie;
13 import com.project.repository.ProjektRepository;
14 import com.project.repository.ZadanieRepository;
15
16 @Service
17 public class ProjektServiceImpl implements ProjektService {
18
19     private static final Logger logger = LoggerFactory.getLogger(ProjektServiceImpl.class);
20
21     private final ProjektRepository projektRepository;
22     private final ZadanieRepository zadanieRepository;
23
24     @Autowired // w tej wersji konstruktora Spring wstrzyknie dwa repozytoria
25     public ProjektServiceImpl(ProjektRepository projektRepository, ZadanieRepository zadanieRepository) {
26         this.projektRepository = projektRepository;
27         this.zadanieRepository = zadanieRepository;
28     }
29
30     @Override
31     public Optional<Projekt> getProjekt(Integer projektId) {
32         return projektRepository.findById(projektId);
33     }
34
35     @Override
36     public Projekt setProjekt(Projekt projekt) {
37         return projektRepository.save(projekt);
38     }
39
40     @Override
41     @Transactional
42     public void deleteProjekt(Integer projektId) {
43         for (Zadanie zadanie : zadanieRepository.findZadaniaProjektu(projektId)) {
44             zadanieRepository.delete(zadanie);
45         }
46         projektRepository.deleteById(projektId);
47     }
48
49     @Override
50     public Page<Projekt> getProjekty(Pageable pageable) {
51         return projektRepository.findAll(pageable);
52     }
53
54     @Override
55     public Page<Projekt> searchByNazwa(String nazwa, Pageable pageable) {
56         return projektRepository.findByNazwaContainingIgnoreCase(nazwa, pageable);
57     }
58 }

```

Screen 7 - klasa ProjektServiceImpl.java z implementacją transakcyjnej metody usuwania projektu

2.8. Kontrolery REST - klasa ProjektRestController

Klasę `ProjektRestController` w pakiecie `com.project.controller` oznaczyłem adnotacjami `@RestController` oraz `@RequestMapping('/api')`. Adnotacja `@RestController` łączy działanie adnotacji `@Controller` oraz `@ResponseBody`, informując że zwracane obiekty są przekształcane w strumień danych JSON i przesyłane w ciele odpowiedzi HTTP. Za pomocą adnotacji `@Tag(name='Projekt')` zintegrowałem kontroler z generatorem dokumentacji OpenAPI.

W klasie zaimplementowałem następujące endpointy realizujące operacje CRUD:

1. GET `/api/projekty/{projectId}` - pobieranie pojedynczego projektu. Wykorzystuje adnotację `@PathVariable` do mapowania zmiennej z adresu URI. Zwraca obiekt opakowany w `ResponseEntity`, w przypadku braku rekordu zwraca to kod odpowiedzi 404 Not Found.
2. POST `/api/projekty` - tworzenie zasobu. Dane projektu są przekazywane w ciele żądania (adnotacja `@RequestBody`) i walidowane przez `@Valid`. Po udanym zapisie, przy użyciu klasy `ServletUriComponentsBuilder`, generowany jest pełny link URI do nowo utworzonego zasobu, wstrzykiwany do nagłówka `Location` odpowiedzi i zwracany status 201 Created.
3. PUT `/api/projekty/{projectId}` - aktualizacja danych. Weryfikuje czy zasób istnieje, wykonuje zapis i zwraca kod statusu 200 OK.
4. DELETE `/api/projekty/{projectId}` - usuwanie zasobu, zwraca status 200 OK lub 404 Not Found.

Metoda `getProjekty` przyjmuje obiekt `Pageable` opatrzony adnotacją `@ParameterObject` biblioteki `SpringDoc`, która rozkłada złożony obiekt `Pageable` na pojedyncze parametry zapytania URL (`?page=0&size=10&sort=nazwa,desc`) wyświetlając je w ten sposób w dokumentacji Swagger. Ponadto, używając parametru `params='nazwa'` dla mapowania, zaimplementowałem metodę `getProjektyByNazwa`, wywoływaną wtedy, gdy w adresie zostanie przekazana fraza wyszukiwania.

```
ProjektRestController.java ×
9 import org.springframework.data.domain.Pageable;
10 import org.springframework.http.HttpStatus;
11 import org.springframework.http.ResponseEntity;
12 import org.springframework.web.bind.annotation.*;
13 import org.springframework.web.servlet.support.ServletUriComponentsBuilder;
14 import com.project.model.Projekt;
15 import com.project.service.ProjektService;
16 import org.springdoc.core.annotations.ParameterObject;
17 import io.swagger.v3.oas.annotations.tags.Tag;
18
19 @RestController
20 // dzięki adnotacji @RestController klasa jest traktowana jako zarządzany
21 // przez kontener Springa REST-owy kontroler obsługujący sieciowe żądania
22 @RequestMapping("/api")
23 // adnotacja @RequestMapping umieszczona w tym miejscu pozwala definiować
24 // część wspólną adresu, wstawiana przed wszystkimi poniższymi ścieżkami
25 @Tag(name = "Projekt") // zmiana nazwy, uwzględniania min. przy generowaniu specyfikacji za pomocą OpenAPI
26 public class ProjektRestController {
27
28     private static final Logger logger = LoggerFactory.getLogger(ProjektRestController.class);
29
30     private final ProjektService projektService; // serwis jest automatycznie wstrzykiwany poprzez konstruktor
31
32     // @Autowired opcjonalne, potrzebne tylko przy kilku konstruktorach
33     public ProjektRestController(ProjektService projektService) {
34         this.projektService = projektService;
35     }
36
37     // PRZED KAŻDĄ Z PONIŻSZYCH METOD JEST UMIESZCZONA ADNOTACJA (@GetMapping, PostMapping, ), KTÓRA OKREŚLA
38     // RODZAJ METODY HTTP, A TAKŻE ADRES I PARAMETRY ŻĄDANIA
39     // Przykład żądania wywołującego metodę: GET http://localhost:8080/api/projekty/1
40     @GetMapping("/projekty/{projektId}")
41     public ResponseEntity<Projekt> getProjekt(@PathVariable("projektId") Integer projektId) {
42         // @PathVariable oznacza, że wartość parametru przekazywana jest w ścieżce
43         return ResponseEntity.of(projektService.getProjekt(projektId));
44     }
45
46     // @Valid włącza automatyczną walidację na podstawie adnotacji zawartych
47     // w modelu np. NotNull, Size, NotEmpty itd. (z jakarta.validation.constraints.*)
48     @PostMapping(path = "/projekty")
49     public ResponseEntity<Void> createProjekt(@Valid @RequestBody Projekt projekt) {
50         // @RequestBody oznacza, że dane projektu (w formacie JSON) są przekazywane w ciele żądania
51         Projekt createdProjekt = projektService.setProjekt(projekt);
52
53         URI location = ServletUriComponentsBuilder.fromCurrentRequest().buildAndExpand(createdProjekt.getId()).toUri();
54
55         return ResponseEntity.created(location).build(); // zwracany jest kod odpowiedzi 201 Created z linkiem location w nagłówku
56     }
57
58     @PutMapping("/projekty/{projektId}")
59     public ResponseEntity<Void> updateProjekt(@Valid @RequestBody Projekt projekt, @PathVariable("projektId") Integer projektId) {
60         return projektService.getProjekt(projektId)
61             .map(p -> {
62                 projektService.setProjekt(projekt);
63                 return new ResponseEntity<Void>(HttpStatus.OK); // 200 (można też zwracać 204 No content)
64             })
65             .orElseGet(() -> ResponseEntity.notFound().build()); // 404 Not found
66     }
67
68     @DeleteMapping("/projekty/{projektId}")
69     public ResponseEntity<Void> deleteProjekt(@PathVariable("projektId") Integer projektId) {
70         return projektService.getProjekt(projektId).map(p -> {
71             projektService.deleteProjekt(projektId);
72             return new ResponseEntity<Void>(HttpStatus.OK); // 200
73         }).orElseGet(() -> ResponseEntity.notFound().build()); // 404 Not found
74     }
75
76     // Przykład żądania wywołującego metodę: http://localhost:8080/api/projekty?page=0&size=10&sort=nazwa, desc
77     @GetMapping(value = "/projekty")
78     public Page<Projekt> getProjekty(@ParameterObject Pageable pageable) {
79         // jeżeli potrzebny byłby nagłówek, wystarczy dodać drugą zmienną z adnotacją @RequestHeader HttpHeaders headers
80         // @ParameterObject wykorzystywany przez OpenAPI pozwala rozbić obiekt Pageable
81         // na pojedyncze parametry (zawarte w tym obiekcie) URL i traktuje je jako opcjonalne
82         return projektService.getProjekty(pageable);
83     }
84
85     // Przykład żądania wywołującego metodę: GET http://localhost:8080/api/projekty?nazwa=webowa
86     // Metoda zostanie wywołana tylko, gdy w żądaniu będzie przesłana wartość parametru nazwa.
87     @GetMapping(value = "/projekty", params = "nazwa")
88     public Page<Projekt> getProjektyByNazwa(@RequestParam(name = "nazwa") String nazwa, @ParameterObject Pageable pageable) {
89         return projektService.searchByNazwa(nazwa, pageable);
90     }
91 }
92 }
```

Screen 8 - kod źródłowy kontrolera ProjektRestController.java z obsługą metod HTTP i walidacją żądań

2.9. Konfiguracja mechanizmu rejestracji (w pliku logback-spring.xml)

W pliku logback-spring.xml w katalogu src/main/resources skonfigurowałem rejestrację logów. Zastępuje on standardowe formatowanie konsoli i definiuje nowe wyjścia komunikatów: ConsoleAppender (STDOUT) drukuje sformatowane informacje w Eclipse, natomiast RollingFileAppender (FILE) zapisuje logi do pliku project-application.log w katalogu logs/.

Polityka rotacji plików to TimeBasedRollingPolicy. Każdego dnia o północy aktualny log jest zamykany, kompresowany i archiwizowany (ZIP), a następnie przenoszony do podfolderu archive/. Skonfigurowany limit retencji maxHistory wynosi 30 dni, a totalSizeCap ustawiłem na 30MB. Poziom logowania dla pakietu głównego com.project jest ustawiony na INFO, i to pozwala odfiltrowywać zbędne komunikaty diagnostyczne.

2.10. Testowanie REST API za pomocą programu Postman

Po uruchomieniu aplikacji, ręcznie przetestowałem punkty końcowe za pomocą narzędzia Postman. Usługa została zabezpieczona przez Spring Security, dlatego pierwsze próby wywołania endpointów bez poświadczeń kończyły się odrzuceniem żądania i zwróceniem kodu błędu 401 Unauthorized. W zakładce 'Authorization' programu Postman wybrałem typ 'Basic Auth', wprowadzając zdefiniowany login oraz hasło.

Pierwszym testem było wysłanie żądania typu POST pod adres `http://localhost:8080/api/projekty`, żeby w bazie PostgreSQL pojawiły się testowe rekordy. W zakładce 'Body' zazaczyłem opcję 'raw' oraz format 'JSON', wprowadzając strukturę reprezentującą nowy projekt. Po kliknięciu przycisku 'Send', usługa zapisała rekord w bazie generując unikalny identyfikator i zwróciła status 201 Created. W sekcji nagłówków odpowiedzi zauważyłem obecność nagłówka Location wskazującego adres zasobu. Następnie przetestowałem żądanie GET, które pobierało listę z dodatkowymi parametrami stronicowania (`?page=0&size=5&sort=nazwa,desc`), analizowało otrzymaną strukturę JSON z metadanymi paginacji `totalPages` i `totalElements`.

Home Workspaces Explore Search Postman Sign In Create Account

You are using the Lightweight API Client, sign in or create an account to work with collections, environments and unlock all free features in Postman.

History New Import POST http://localhost:8080/api/projekty GET http://localhost:8080/api/projekty GET http://localhost:8080/api/projekty/1 PUT http://localhost:8080/api/projekty/1 POST http://localhost:8080/api/projekty/1

Today
POST http://localhost:8080/api/projekty

July 7
POST http://localhost:8080/api/projekty

June 30
GET http://localhost:8080/api/projekty
PUT http://localhost:8080/api/projekty/1
GET http://localhost:8080/api/projekty/1
GET http://localhost:8080/api/projekty
GET http://localhost:8080/api/projekty
POST http://localhost:8080/api/projekty

HTTP http://localhost:8080/api/projekty Save </>

POST http://localhost:8080/api/projekty Send

Params Auth Headers (9) Body Pre-req. Tests Settings Cookies Beautify

raw JSON

```

1
2 "nazwa": "Nowy projekt testowy REST",
3 "opis": "Opis projektu testowego REST",
4 "dataOddania": "2026-01-01"
5

```

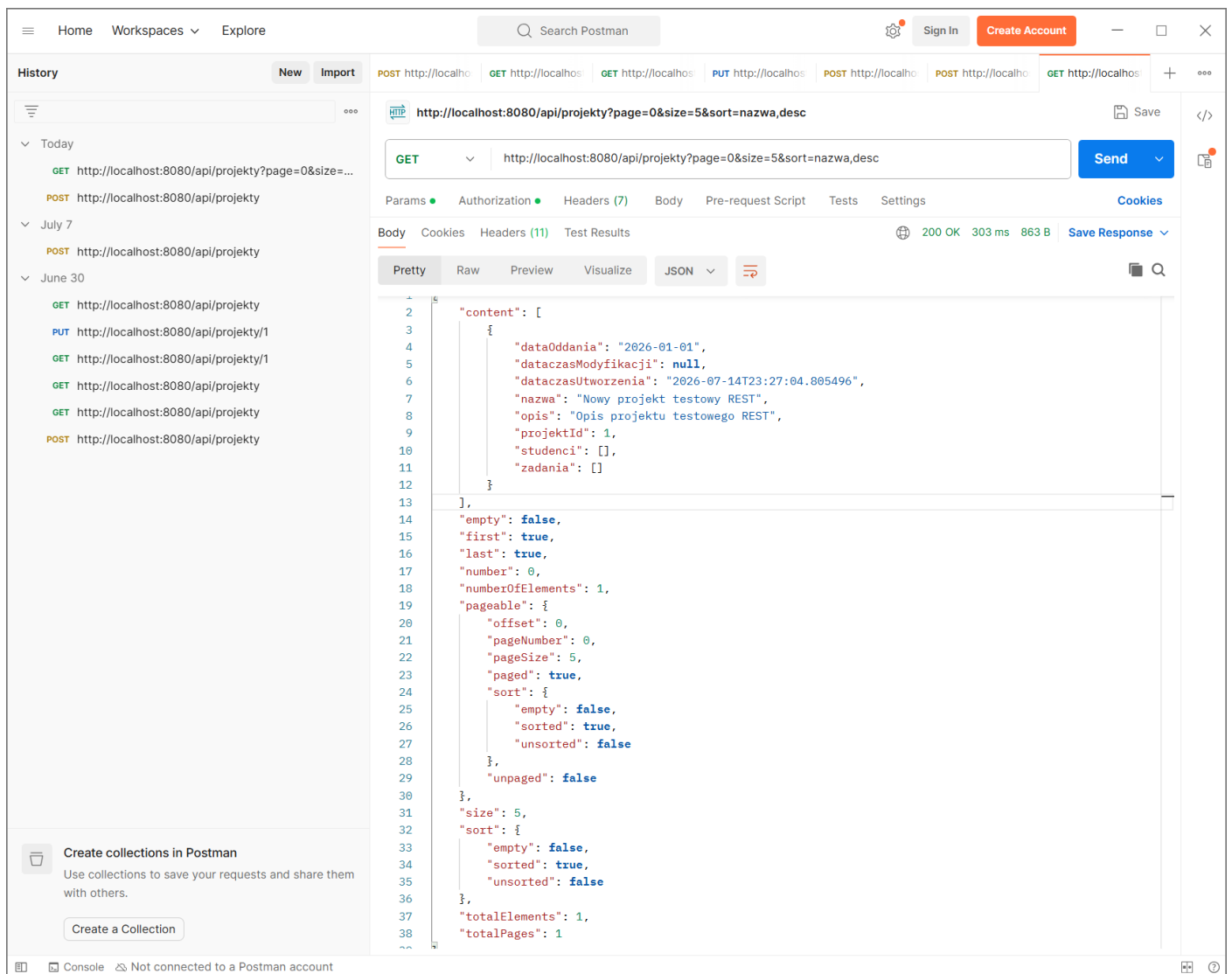
Headers 201 Created 843 ms 346 B Save Response

Key	Value
Location	http://localhost:8080/api/projekty/1
X-Content-Type-Options	nosniff
X-XSS-Protection	0
Cache-Control	no-cache, no-store, max-age=0, must-revalidate
Pragma	no-cache
Expires	0
X-Frame-Options	DENY
Content-Length	0

Create collections in Postman
Use collections to save your requests and share them with others.
Create a Collection

Console Not connected to a Postman account

Screen 9 - okno programu Postman: wysłanie żądania POST z nagłówkiem Basic Auth i ciałem JSON w celu utworzenia nowego projektu; widoczny status 201 Created oraz wiersz z nagłówkiem Location w tabeli poniżej

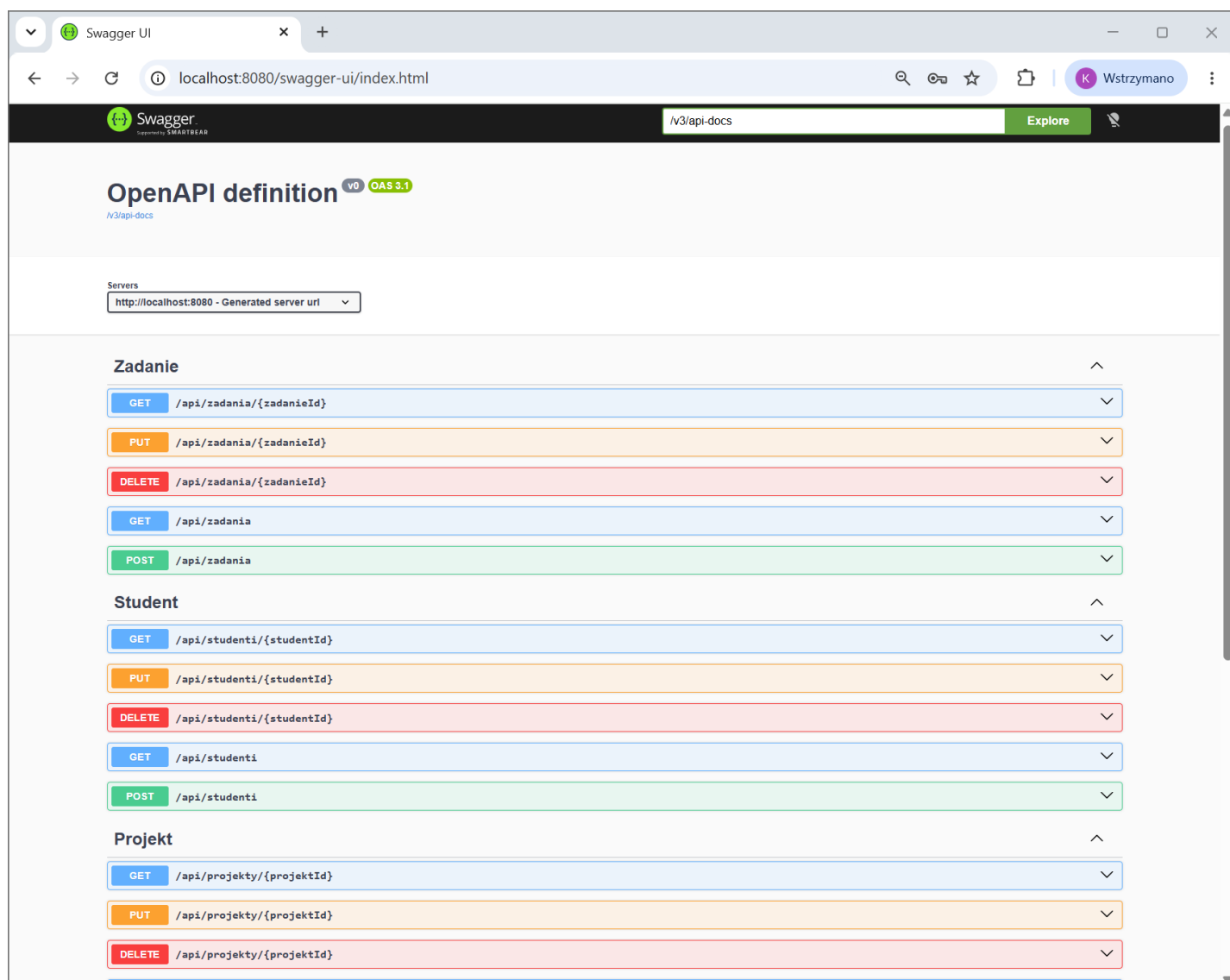


Screen 10 - okno programu Postman: odpowiedź na żądanie GET jako sformatowana lista zasobów oraz struktura stronicowania bazy danych

2.11. Dokumentacja usługi za pomocą interfejsu Swagger UI

Do pliku konfiguracyjnego `build.gradle` dodałem zależność `org.springdoc:springdoc-openapi-starter-webmvc-ui:3.0.2`, aby zautomatyzować generowanie dokumentacji technicznej API. Ta biblioteka skanuje wszystkie klasy oznaczone adnotacją `@RestController` podczas startu aplikacji. Analizuje mapowania metod, typy przyjmowanych parametrów oraz zwracane kody odpowiedzi, następnie tworzy strukturalny opis API według standardu OpenAPI 3.

Interfejs dokumentacji jest dostępny pod adresem <http://localhost:8080/swagger-ui/index.html>. Po załadowaniu strony w przeglądarce wyświetla się menu grupujące endpointy według zasobów (Projekt, Zadanie, Student). Każdy endpoint można rozwinąć, analizując strukturę obiektu JSON oraz schematy walidacyjne pól. Swagger UI udostępnia również funkcjonalność 'Try it out', wykonując bezpośrednio zapytania testowe (POST, GET, PUT, DELETE) z poziomu przeglądarki internetowej, wyświetla konkretne nagłówki odpowiedzi oraz wygenerowane polecenia curl.



Screen 11 - główny widok interfejsu Swagger UI w przeglądarce z listą wygenerowanych endpointów REST API dla zasobów

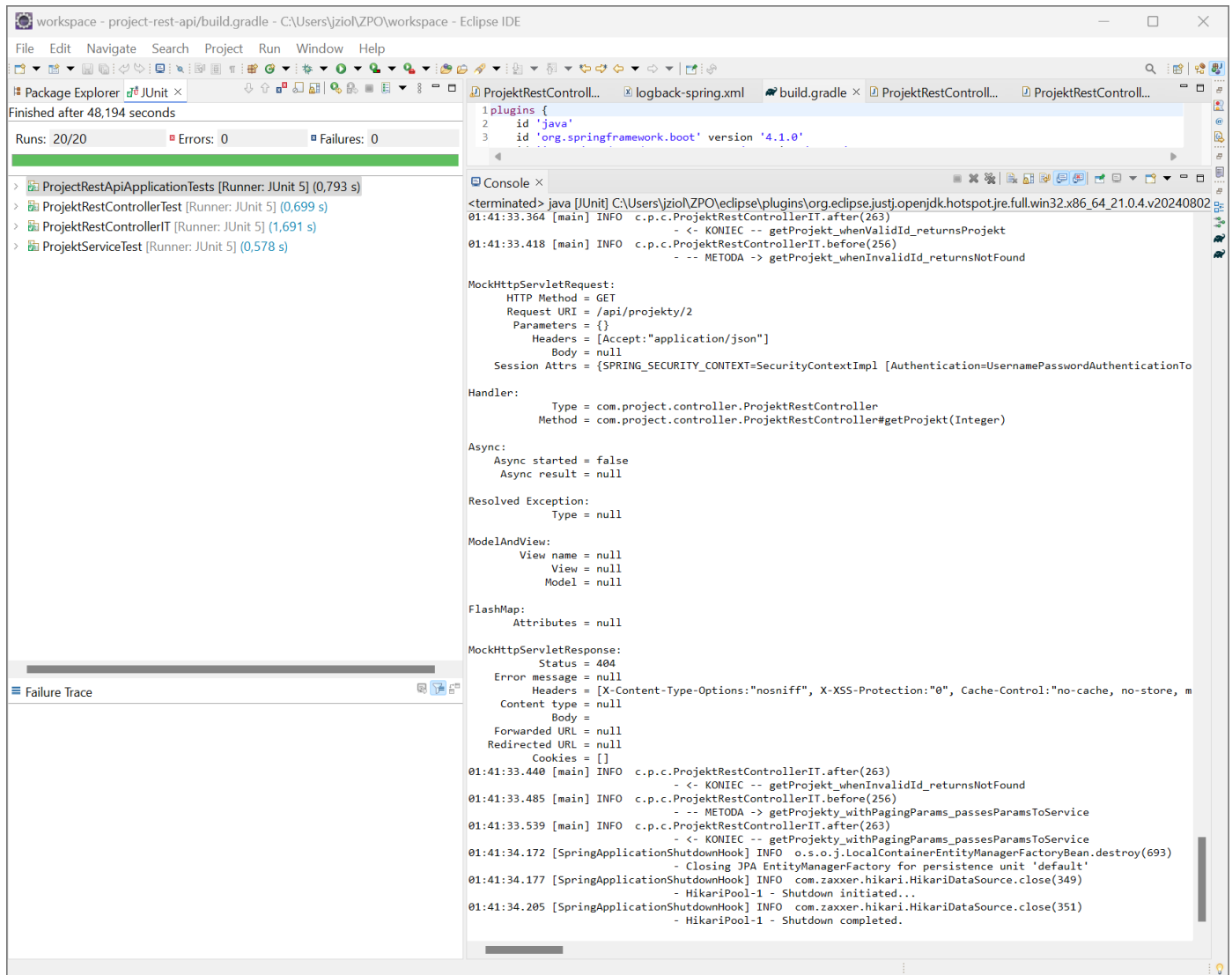
2.12. Testy integracyjne i jednostkowe

Proces testowy zautomatyzowałem przy użyciu frameworka JUnit 5 oraz bibliotek MockMvc i Mockito. W katalogu `src/test/java` w pakiecie `com.project.controller` zaimplementowałem dwie klasy testowe: `ProjektRestControllerIT` (testy integracyjne warstwy webowej) oraz `ProjektRestControllerTest` (testy jednostkowe w izolacji).

W klasie `ProjektRestControllerIT` zastosowałem adnotacje: `@WebMvcTest(ProjektRestController.class)` oraz `@AutoConfigureJsonTesters`. Adnotacja `@WebMvcTest` instruuje Springa, aby inicjalizował tylko warstwę sieciową powiązaną ze wskazanym kontrolerem REST. Adnotacja `@WithMockUser(username='admin', password='admin')` symuluje uwierzytelnionego użytkownika.

Z kolei adnotacja `@MockitoBean` pozwala utworzyć mock zależności serwisowej jako `mockProjektService`. Wykorzystując `MockMvc`, zaimplementowałem metody testowe wykonujące wirtualne żądania HTTP. Asercja `andExpect` sprawdza wyniki, testuje statusy odpowiedzi oraz poprawność struktury JSON przy użyciu ścieżek `jsonPath`. Przetestowałem również przypadek przesłania niepoprawnej, pustej nazwy (zwracany kod 400 Bad Request).

Zaimplementowałem też klasę `ProjektRestControllerTest` opartą na rozszerzeniu `@ExtendWith(MockitoExtension.class)`. Ten test nie uruchamia żadnych mechanizmów Spring Boot, dzięki temu wykonuje się w kilka milisekund i testuje wyłącznie logikę Javy. Zależności wstrzykiwane są za pomocą adnotacji `@Mock` oraz `@InjectMocks`. Problemy z kodowaniem polskich znaków diakrytycznych znikły, gdy w pliku `build.gradle` wymusiłem kodowanie UTF-8 dla zadań `compileJava` oraz `compileTestJava`.



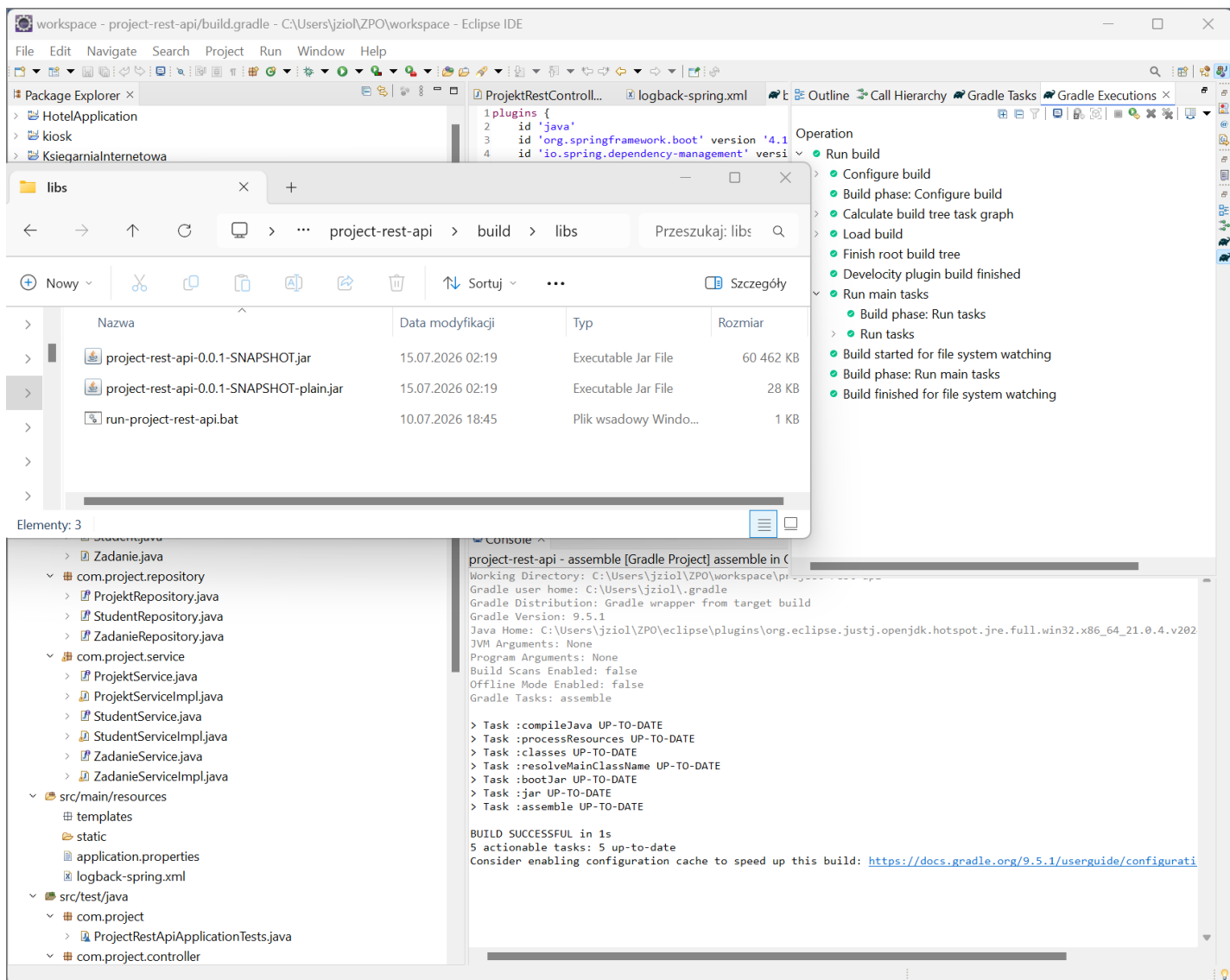
Screen 12 - w widoku JUnit w Eclipse IDE widoczny zielony pasek sukcesu i poprawne wykonanie wszystkich testów

2.13. Tworzenie wersji uruchomieniowej i automatyzacja uruchamiania

Ostatnim etapem było przygotowanie wersji dystrybucyjnej usługi Rest API. Uruchomiłem zadanie systemu Gradle o nazwie 'assemble' z poziomu dedykowanego widoku Gradle Tasks w Eclipse, tym samym inicjując proces kompilacji kodu źródłowego, uruchomienia testów oraz spakowania aplikacji ze wszystkimi zależnościami do pliku archiwum project-rest-api-0.0.1-SNAPSHOT.jar, zlokalizowanego w podkatalogu build/libs.

Utworzyłem w systemie plik wsadowy run-project-rest-api.bat, żeby nie musieć za każdym razem otwierać Eclipse IDE przy uruchamianiu usługi. W tym skrypcie wskazałem ścieżkę bezwzględną do katalogu bin dystrybucji OpenJDK 21 osadzoną w pluginach Eclipse (instrukcją 'set path'), a następnie zdefiniowałem polecenie uruchomieniowe 'java -jar project-rest-api-0.0.1-SNAPSHOT.jar'.

Wykonanie pliku .bat w konsoli cmd powoduje uruchomienie Spring Boot i bezkierunkowej usługi REST. Dodatkowym atutem tego rozwiązania jest możliwość bezpiecznego zamykania aplikacji skrótem Ctrl+C.



Screen 13 - widok zakładki Gradle Executions w Eclipse po wykonaniu assemble w Gradle Tasks, a także zawartość katalogu build/libs z plikiem JAR

```
Administrator: Wiersz poleceń X + v
C:\Users\jzliol>cd C:\Users\jzliol\ZPO\workspace\project-rest-api\build\libs
C:\Users\jzliol\ZPO\workspace\project-rest-api\build\libs>run-project-rest-api.bat
C:\Users\jzliol\ZPO\workspace\project-rest-api\build\libs>set path="c:\Users\jzliol\ZPO\eclipse\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.4.v20240802-1551\jre\bin\"
C:\Users\jzliol\ZPO\workspace\project-rest-api\build\libs>java -jar project-rest-api-0.0.1-SNAPSHOT.jar
02:32:33.745 -INFO in ch.qos.logback.classic.model.processor.ConfigurationModelHandlerFull - Scan attribute not set or set to unrecognized value.
02:32:33.749 -INFO in ch.qos.logback.core.model.processor.ModelInterpretationContext@6256ac4f - value "logs/archive" substituted for "${LOG_DIR}/archive"
02:32:33.752 -INFO in ch.qos.logback.core.model.processor.ModelInterpretationContext@6256ac4f - value "logs/project-application.log" substituted for "${LOG_DIR}/${LOG_FILE}.log"
02:32:33.753 -INFO in ch.qos.logback.core.model.processor.ModelInterpretationContext@6256ac4f - value "logs/archive/${yyyy-MM-dd}project-application.log.zip" substituted for "${LOG_ARCHIVE}/${yyyy-MM-dd}${LOG_FILE}.log.zip"
02:32:33.755 -INFO in ch.qos.logback.core.model.processor.AppenderModelHandler - Processing appender named [STDOUT]
02:32:33.755 -INFO in ch.qos.logback.core.model.processor.AppenderModelHandler - About to instantiate appender of type [ch.qos.logback.core.ConsoleAppender]
02:32:33.762 -INFO in ch.qos.logback.core.model.processor.ImplicitModelHandler - Assuming default type [ch.qos.logback.classic.encoder.PatternLayoutEncoder] for [encoder] property
02:32:33.812 -INFO in ch.qos.logback.core.ConsoleAppender[STDOUT] - NOTE: Writing to the console can be slow. Try to avoid logging to the
02:32:33.812 -INFO in ch.qos.logback.core.ConsoleAppender[STDOUT] - console in production environments, especially in high volume systems.
02:32:33.812 -INFO in ch.qos.logback.core.model.processor.AppenderModelHandler - Processing appender named [FILE]
02:32:33.813 -INFO in ch.qos.logback.core.model.processor.AppenderModelHandler - About to instantiate appender of type [ch.qos.logback.core.rolling.RollingFileAppender]
02:32:33.816 -INFO in ch.qos.logback.core.model.processor.ModelInterpretationContext@6256ac4f - value "logs/project-application.log" substituted for "${LOG_DIR}/${LOG_FILE}.log"
02:32:33.820 -INFO in ch.qos.logback.core.model.processor.ModelInterpretationContext@6256ac4f - value "logs/archive/${yyyy-MM-dd}project-application.log.zip" substituted for "${LOG_ARCHIVE}/${yyyy-MM-dd}${LOG_FILE}.log.zip"
02:32:33.826 -INFO in c.q.l.core.rolling.TimeBasedRollingPolicy@2115640742 - setting totalSizeCap to 30 MB
02:32:33.827 -INFO in c.q.l.core.rolling.TimeBasedRollingPolicy@2115640742 - Will use zip compression
02:32:33.831 -INFO in c.q.l.core.rolling.TimeBasedRollingPolicy@2115640742 - Will use the pattern logs/archive/${yyyy-MM-dd}project-application.log for the active file
02:32:33.887 -INFO in c.q.l.core.rolling.DefaultTimeBasedFileNamingAndTriggeringPolicy - Roll-over at midnight.
02:32:33.890 -INFO in c.q.l.core.rolling.DefaultTimeBasedFileNamingAndTriggeringPolicy - Setting initial period to 2026-07-15T00:32:33.889Z
02:32:33.892 -INFO in ch.qos.logback.classic.model.processor.AppenderRefModelHandler - Assuming default type [ch.qos.logback.classic.encoder.PatternLayoutEncoder] for [encoder] property
02:32:33.895 -INFO in ch.qos.logback.core.rolling.RollingFileAppender[FILE] - Active log file name: logs/project-application.log
02:32:33.895 -INFO in ch.qos.logback.core.rolling.RollingFileAppender[FILE] - File property is set to [logs/project-application.log]
02:32:33.898 -INFO in ch.qos.logback.classic.model.processor.LoggerModelHandler - Setting level of logger [com.project] to INFO
02:32:33.899 -INFO in ch.qos.logback.classic.jul.LevelChangePropagator@6b9267b - Propagating INFO level on Logger[com.project] onto the JUL framework
02:32:33.908 -INFO in ch.qos.logback.classic.model.processor.LoggerModelHandler - Setting additivity of logger [com.project] to false
02:32:33.908 -INFO in ch.qos.logback.classic.model.processor.AppenderRefModelHandler - Attaching appender named [STDOUT] to Logger[com.project]
02:32:33.909 -INFO in ch.qos.logback.classic.model.processor.AppenderRefModelHandler - Attaching appender named [FILE] to Logger[com.project]
02:32:33.909 -INFO in ch.qos.logback.classic.model.processor.RootLoggerModelHandler - Setting level of ROOT logger to INFO
02:32:33.909 -INFO in ch.qos.logback.classic.jul.LevelChangePropagator@6b9267b - Propagating INFO level on Logger[ROOT] onto the JUL framework
02:32:33.909 -INFO in ch.qos.logback.core.model.processor.AppenderRefModelHandler - Attaching appender named [STDOUT] to Logger[ROOT]
02:32:33.910 -INFO in ch.qos.logback.core.model.processor.DefaultProcessorCfScale - End of configuration.
02:32:33.910 -INFO in org.springframework.boot.logging.logback.SpringBootJoranConfigurator@241a53ef - Registering current configuration as safe fallback point

=====
:: Spring Boot :: (v0.1.0)

02:32:33.992 [background-preinit] INFO o.h.validator.internal.util.Version.<clinit>(18)
- HW000001: Hibernate Validator 9.1.0.Final
02:32:34.083 [main] INFO c.project.ProjectRestApiApplication.logStarting(54)
- Starting ProjectRestApiApplication v0.0.1-SNAPSHOT using Java 21.0.4 with PID 15812 (C:\Users\jzliol\ZPO\workspace\project-rest-api\build\libs\project-rest-api-0.0.1-SNAPSHOT.jar started by kziołkowski in C:\Users\jzliol\ZPO\workspace\project-rest-api\build\libs)
02:32:34.085 [main] INFO c.project.ProjectRestApiApplication.logStartupProfileInfo(652)
- No active profile set, falling back to 1 default profile: "default"
02:32:35.337 [main] INFO o.s.d.r.c.RepositoryConfigurationDelegate.registerRepositoriesIn(145)
- Bootstrapping Spring Data JPA repositories in DEFAULT mode.
02:32:35.384 [main] INFO o.s.d.r.c.RepositoryConfigurationDelegate.registerRepositoriesIn(213)
- Finished Spring Data repository scanning in 37 ms. Found 3 JPA repository interfaces.
02:32:36.450 [main] INFO o.s.boot.tomcat.TomcatWebServer.initialize(114)
- Tomcat initialized with port 8080 (http)
02:32:36.476 [main] INFO o.a.coyote.http11.Http11NioProtocol.log(168)
- Initializing ProtocolHandler ["http-nio-8080"]
02:32:36.482 [main] INFO o.a.catalina.core.StandardService.log(168)
- Starting service [Tomcat]
02:32:36.482 [main] INFO o.a.catalina.core.StandardEngine.log(168)
- Starting Servlet engine: [Apache Tomcat/11.0.22]
02:32:36.540 [main] INFO o.s.b.w.e.s.WebApplicationContextInitializer.prepareWebApplicationContext(77)
- Root WebApplicationContext: initialization completed in 2385 ms
02:32:36.857 [main] INFO org.hibernate.orm.jpa.LogPersistenceUnitInformation(182)
- HH0008540: Processing PersistenceUnitInfo [name: default]
02:32:36.930 [main] INFO org.hibernate.orm.core.logVersion(41)
- HH0000001: Hibernate ORM core version 7.4.1.Final
02:32:37.701 [main] INFO o.s.o.j.p.SpringPersistenceUnitInfo.addTransformer(129)
- No LoadTimeWeaver setup: ignoring JPA class transformer
```

Screen 14 - okno systemowego Wiersza polecenia (cmd) po poprawnym uruchomieniu skryptu run-project-rest-api.bat i starcie serwera

3. Wnioski końcowe/Podsumowanie

Realizacja czwartego laboratorium z przedmiotu Zaawansowane Programowanie Obiektowe obejmowała czynności projektowania, implementacji, zabezpieczania oraz testowania usług sieciowych typu REST API przy użyciu platformy Spring Boot oraz bazy danych PostgreSQL.

Na podstawie wykonanych ćwiczeń zweryfikowałem nieocenioną rolę, jaką odgrywa Spring Data JPA. Generując warstwę persystencji na podstawie interfejsów JpaRepository, skraca czas programowania operacji bazodanowych, jednocześnie zapewniając bezpieczeństwo przed atakami typu SQL Injection dzięki sparametryzowaniu zapytań.

Niezwykle istotnym poznanym elementem okazały się wątki wirtualne. Aktywacja parametru `spring.threads.virtual.enabled` sprawia, że synchroniczny serwer Tomcat zyskuje cechy reaktywności i asynchroniczności, a operacje blokujące (takie jak oczekiwanie na serwer PostgreSQL) nie marnują już zasobów procesora. Dzięki temu zwiększyłem przepustowość serwera bez konieczności dokonywania innych modyfikacji w kodzie warstwy biznesowej aplikacji.

Zastosowałem moduł Spring Security w celu zabezpieczenia punktów końcowych aplikacji. Biblioteka SpringDoc OpenAPI pomogła utworzyć dokumentację techniczną, przydatną z Swagger UI. Zautomatyzowane testy integracyjne z MockMVC zagwarantowały poprawność mapowań czy Bean Validation. Całość prac wykonanych podczas przygotowywania tego sprawozdania była zgodna z założeniami instrukcji laboratoryjnej i pozwoliła na wygenerowanie stabilnego i skalowalnego projektu.